EXPERIMENT

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Name **:** S.Nagulan

Reg No : 1924260216

(This experiment is divided into 5 problems)

Problem 1: EnjoySport Data Set

Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis consistent with the given training samples of the enjoysport data set, which decides whether a person enjoys a sport on a given day.

Procedure

1. Define the 6 attributes (Sky, AirTemp, Humidity, Wind, Water, Forecast) and store the training examples, each labelled Yes (positive) or No (negative), in a list.
2. Initialise the hypothesis h with the most specific value, h = ['0', '0', '0', '0', '0', '0'].
3. Take the training examples one by one and ignore every negative example (FIND-S considers positive examples only).
4. For each positive example, compare every attribute of h with the example: if h has '0', replace it with the attribute value of the example.
5. If the attribute value of h differs from the example, generalise that attribute by replacing it with '?'; otherwise keep it unchanged.
6. After all examples are processed, print the hypothesis after each instance and display the final most specific hypothesis.

Coding (Python)

```
# FIND-S Algorithm - EnjoySport Data Set

attributes = ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']

training_data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes'],
]

def find_s(attributes, data):
    n = len(attributes)
    h = ['0'] * n                    # most specific hypothesis
    print("Initial hypothesis :", h)
    print()
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":           # consider positive examples only
            for j in range(n):
                if h[j] == '0':
                    h[j] = x[j]      # first positive example
                elif h[j] != x[j]:
                    h[j] = '?'       # generalise the attribute
        print("Instance", i, "(" + label + ") ->", h)
    return h

print("Training Examples:")
for row in training_data:
    print(row)
```

```
print()
final_h = find_s(attributes, training_data)
print()
print("Most specific hypothesis :", final_h)
```

Input

Training data set (4 examples, 6 attributes, target attribute: EnjoySport):

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Output

```
Training Examples:
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Initial hypothesis : ['0', '0', '0', '0', '0', '0']

Instance 1 (Yes) -> ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
Instance 2 (Yes) -> ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
Instance 3 (No) -> ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
Instance 4 (Yes) -> ['Sunny', 'Warm', '?', 'Strong', '?', '?']

Most specific hypothesis : ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result

Thus the FIND-S algorithm was implemented and demonstrated successfully. The most specific hypothesis for the enjoysport data set is **['Sunny', 'Warm', '?', 'Strong', '?', '?']**.

Problem 2: Weather (Play Tennis) Data Set

Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis consistent with the given training samples of the weather (play tennis) data set, which decides whether tennis can be played under given weather conditions.

Procedure

1. Define the 4 attributes (Outlook, Temperature, Humidity, Wind) and store the training examples, each labelled Yes (positive) or No (negative), in a list.
2. Initialise the hypothesis h with the most specific value, h = ['0', '0', '0', '0'].
3. Take the training examples one by one and ignore every negative example (FIND-S considers positive examples only).
4. For each positive example, compare every attribute of h with the example: if h has '0', replace it with the attribute value of the example.
5. If the attribute value of h differs from the example, generalise that attribute by replacing it with '?'; otherwise keep it unchanged.
6. After all examples are processed, print the hypothesis after each instance and display the final most specific hypothesis.

Coding (Python)

```
# FIND-S Algorithm - Weather (Play Tennis) Data Set

attributes = ['Outlook', 'Temperature', 'Humidity', 'Wind']

training_data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Rainy', 'Mild', 'High', 'Strong', 'No'],
    ['Overcast', 'Mild', 'High', 'Weak', 'Yes'],
]

def find_s(attributes, data):
    n = len(attributes)
    h = ['0'] * n                    # most specific hypothesis
    print("Initial hypothesis :", h)
    print()
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":           # consider positive examples only
            for j in range(n):
                if h[j] == '0':
                    h[j] = x[j]      # first positive example
                elif h[j] != x[j]:
                    h[j] = '?'       # generalise the attribute
        print("Instance", i, "(" + label + ") ->", h)
    return h

print("Training Examples:")
for row in training_data:
```

```
    print(row)
print()
final_h = find_s(attributes, training_data)
print()
print("Most specific hypothesis :", final_h)
```

Input

Training data set (5 examples, 4 attributes, target attribute: PlayTennis):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Overcast	Hot	High	Weak	Yes
Overcast	Cool	Normal	Strong	Yes
Rainy	Mild	High	Strong	No
Overcast	Mild	High	Weak	Yes

Output

```
Training Examples:
['Sunny', 'Hot', 'High', 'Weak', 'No']
['Overcast', 'Hot', 'High', 'Weak', 'Yes']
['Overcast', 'Cool', 'Normal', 'Strong', 'Yes']
['Rainy', 'Mild', 'High', 'Strong', 'No']
['Overcast', 'Mild', 'High', 'Weak', 'Yes']

Initial hypothesis : ['0', '0', '0', '0']

Instance 1 (No) -> ['0', '0', '0', '0']
Instance 2 (Yes) -> ['Overcast', 'Hot', 'High', 'Weak']
Instance 3 (Yes) -> ['Overcast', '?', '?', '?']
Instance 4 (No) -> ['Overcast', '?', '?', '?']
Instance 5 (Yes) -> ['Overcast', '?', '?', '?']

Most specific hypothesis : ['Overcast', '?', '?', '?']
```

Result

Thus the FIND-S algorithm was implemented and demonstrated successfully. The most specific hypothesis for the weather (play tennis) data set is **['Overcast', '?', '?', '?']**.

Problem 3: Customer Buys Computer Data Set

Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis consistent with the given training samples of the customer buys computer data set, which decides whether a customer will buy a computer.

Procedure

1. Define the 4 attributes (Age, Income, Student, CreditRating) and store the training examples, each labelled Yes (positive) or No (negative), in a list.
2. Initialise the hypothesis h with the most specific value, h = ['0', '0', '0', '0'].
3. Take the training examples one by one and ignore every negative example (FIND-S considers positive examples only).
4. For each positive example, compare every attribute of h with the example: if h has '0', replace it with the attribute value of the example.
5. If the attribute value of h differs from the example, generalise that attribute by replacing it with '?'; otherwise keep it unchanged.
6. After all examples are processed, print the hypothesis after each instance and display the final most specific hypothesis.

Coding (Python)

```
# FIND-S Algorithm - Customer Buys Computer Data Set

attributes = ['Age', 'Income', 'Student', 'CreditRating']

training_data = [
    ['Youth', 'High', 'Yes', 'Fair', 'Yes'],
    ['Youth', 'High', 'No', 'Excellent', 'No'],
    ['Youth', 'Medium', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Low', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'No', 'Excellent', 'No'],
]

def find_s(attributes, data):
    n = len(attributes)
    h = ['0'] * n                    # most specific hypothesis
    print("Initial hypothesis :", h)
    print()
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":           # consider positive examples only
            for j in range(n):
                if h[j] == '0':
                    h[j] = x[j]      # first positive example
                elif h[j] != x[j]:
                    h[j] = '?'       # generalise the attribute
        print("Instance", i, "(" + label + ") ->", h)
    return h

print("Training Examples:")
for row in training_data:
```

```
    print(row)
print()
final_h = find_s(attributes, training_data)
print()
print("Most specific hypothesis :", final_h)
```

Input

Training data set (5 examples, 4 attributes, target attribute: BuysComputer):

Age	Income	Student	CreditRating	BuysComputer
Youth	High	Yes	Fair	Yes
Youth	High	No	Excellent	No
Youth	Medium	Yes	Fair	Yes
Senior	Low	Yes	Fair	Yes
Senior	Medium	No	Excellent	No

Output

```
Training Examples:
['Youth', 'High', 'Yes', 'Fair', 'Yes']
['Youth', 'High', 'No', 'Excellent', 'No']
['Youth', 'Medium', 'Yes', 'Fair', 'Yes']
['Senior', 'Low', 'Yes', 'Fair', 'Yes']
['Senior', 'Medium', 'No', 'Excellent', 'No']

Initial hypothesis : ['0', '0', '0', '0']

Instance 1 (Yes) -> ['Youth', 'High', 'Yes', 'Fair']
Instance 2 (No) -> ['Youth', 'High', 'Yes', 'Fair']
Instance 3 (Yes) -> ['Youth', '?', 'Yes', 'Fair']
Instance 4 (Yes) -> ['?', '?', 'Yes', 'Fair']
Instance 5 (No) -> ['?', '?', 'Yes', 'Fair']

Most specific hypothesis : ['?', '?', 'Yes', 'Fair']
```

Result

Thus the FIND-S algorithm was implemented and demonstrated successfully. The most specific hypothesis for the customer buys computer data set is **['?', '?', 'Yes', 'Fair']**.

Problem 4: Fruit Ripeness Data Set

Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis consistent with the given training samples of the fruit ripeness data set, which decides whether a fruit is ripe.

Procedure

1. Define the 4 attributes (Color, Size, Texture, Smell) and store the training examples, each labelled Yes (positive) or No (negative), in a list.
2. Initialise the hypothesis h with the most specific value, h = ['0', '0', '0', '0'].
3. Take the training examples one by one and ignore every negative example (FIND-S considers positive examples only).
4. For each positive example, compare every attribute of h with the example: if h has '0', replace it with the attribute value of the example.
5. If the attribute value of h differs from the example, generalise that attribute by replacing it with '?'; otherwise keep it unchanged.
6. After all examples are processed, print the hypothesis after each instance and display the final most specific hypothesis.

Coding (Python)

```
# FIND-S Algorithm - Fruit Ripeness Data Set

attributes = ['Color', 'Size', 'Texture', 'Smell']

training_data = [
    ['Yellow', 'Large', 'Soft', 'Sweet', 'Yes'],
    ['Green', 'Small', 'Hard', 'Sour', 'No'],
    ['Yellow', 'Medium', 'Soft', 'Sweet', 'Yes'],
    ['Yellow', 'Large', 'Soft', 'Sweet', 'Yes'],
    ['Green', 'Large', 'Hard', 'Sweet', 'No'],
]

def find_s(attributes, data):
    n = len(attributes)
    h = ['0'] * n                    # most specific hypothesis
    print("Initial hypothesis :", h)
    print()
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":           # consider positive examples only
            for j in range(n):
                if h[j] == '0':
                    h[j] = x[j]      # first positive example
                elif h[j] != x[j]:
                    h[j] = '?'       # generalise the attribute
        print("Instance", i, "(" + label + ") ->", h)
    return h

print("Training Examples:")
for row in training_data:
```

```
    print(row)
print()
final_h = find_s(attributes, training_data)
print()
print("Most specific hypothesis :", final_h)
```

Input

Training data set (5 examples, 4 attributes, target attribute: Ripe):

Color	Size	Texture	Smell	Ripe
Yellow	Large	Soft	Sweet	Yes
Green	Small	Hard	Sour	No
Yellow	Medium	Soft	Sweet	Yes
Yellow	Large	Soft	Sweet	Yes
Green	Large	Hard	Sweet	No

Output

```
Training Examples:
['Yellow', 'Large', 'Soft', 'Sweet', 'Yes']
['Green', 'Small', 'Hard', 'Sour', 'No']
['Yellow', 'Medium', 'Soft', 'Sweet', 'Yes']
['Yellow', 'Large', 'Soft', 'Sweet', 'Yes']
['Green', 'Large', 'Hard', 'Sweet', 'No']

Initial hypothesis : ['0', '0', '0', '0']

Instance 1 (Yes) -> ['Yellow', 'Large', 'Soft', 'Sweet']
Instance 2 (No) -> ['Yellow', 'Large', 'Soft', 'Sweet']
Instance 3 (Yes) -> ['Yellow', '?', 'Soft', 'Sweet']
Instance 4 (Yes) -> ['Yellow', '?', 'Soft', 'Sweet']
Instance 5 (No) -> ['Yellow', '?', 'Soft', 'Sweet']

Most specific hypothesis : ['Yellow', '?', 'Soft', 'Sweet']
```

Result

Thus the FIND-S algorithm was implemented and demonstrated successfully. The most specific hypothesis for the fruit ripeness data set is **['Yellow', '?', 'Soft', 'Sweet']**.

Problem 5: Loan Approval Data Set

Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis consistent with the given training samples of the loan approval data set, which decides whether a bank loan application is approved.

Procedure

1. Define the 4 attributes (Income, Credit, Employment, Property) and store the training examples, each labelled Yes (positive) or No (negative), in a list.
2. Initialise the hypothesis h with the most specific value, h = ['0', '0', '0', '0'].
3. Take the training examples one by one and ignore every negative example (FIND-S considers positive examples only).
4. For each positive example, compare every attribute of h with the example: if h has '0', replace it with the attribute value of the example.
5. If the attribute value of h differs from the example, generalise that attribute by replacing it with '?'; otherwise keep it unchanged.
6. After all examples are processed, print the hypothesis after each instance and display the final most specific hypothesis.

Coding (Python)

```
# FIND-S Algorithm - Loan Approval Data Set

attributes = ['Income', 'Credit', 'Employment', 'Property']

training_data = [
    ['High', 'Good', 'Salaried', 'Owned', 'Yes'],
    ['Low', 'Poor', 'Unemployed', 'Rented', 'No'],
    ['High', 'Good', 'Salaried', 'Rented', 'Yes'],
    ['High', 'Good', 'Self-Employed', 'Owned', 'Yes'],
    ['Low', 'Good', 'Salaried', 'Owned', 'No'],
]

def find_s(attributes, data):
    n = len(attributes)
    h = ['0'] * n                    # most specific hypothesis
    print("Initial hypothesis :", h)
    print()
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":           # consider positive examples only
            for j in range(n):
                if h[j] == '0':
                    h[j] = x[j]      # first positive example
                elif h[j] != x[j]:
                    h[j] = '?'       # generalise the attribute
        print("Instance", i, "(" + label + ") ->", h)
    return h

print("Training Examples:")
for row in training_data:
```

```
    print(row)
print()
final_h = find_s(attributes, training_data)
print()
print("Most specific hypothesis :", final_h)
```

Input

Training data set (5 examples, 4 attributes, target attribute: Approved):

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Owned	Yes
Low	Poor	Unemployed	Rented	No
High	Good	Salaried	Rented	Yes
High	Good	Self-Employed	Owned	Yes
Low	Good	Salaried	Owned	No

Output

```
Training Examples:
['High', 'Good', 'Salaried', 'Owned', 'Yes']
['Low', 'Poor', 'Unemployed', 'Rented', 'No']
['High', 'Good', 'Salaried', 'Rented', 'Yes']
['High', 'Good', 'Self-Employed', 'Owned', 'Yes']
['Low', 'Good', 'Salaried', 'Owned', 'No']

Initial hypothesis : ['0', '0', '0', '0']

Instance 1 (Yes) -> ['High', 'Good', 'Salaried', 'Owned']
Instance 2 (No) -> ['High', 'Good', 'Salaried', 'Owned']
Instance 3 (Yes) -> ['High', 'Good', 'Salaried', '?']
Instance 4 (Yes) -> ['High', 'Good', '?', '?']
Instance 5 (No) -> ['High', 'Good', '?', '?']

Most specific hypothesis : ['High', 'Good', '?', '?']
```

Result

Thus the FIND-S algorithm was implemented and demonstrated successfully. The most specific hypothesis for the loan approval data set is **['High', 'Good', '?', '?']**.